

SNOWPARK CONTAINER SERVICES OVERVIEW



Agenda

- > Overview
- > Architecture
- > Use Cases
- > Deployment Cycle



Background

Why Snowpark Container Services?

Additional capabilities

Containers, Front end (UI), Web APIs, connectors, custom runtimes, custom libraries, GPUs, customized hardware

Re-platforming cost

Portability & Interoperability are important when moving workloads. Consistent with common cloud native patterns

Data Governance

Data not leaving Snowflake to do processing or advanced compute provides security & governance

Customer opportunity

Managing compute anywhere has high overhead. Containers and compute is complex. Many personas (partners, data scientists, app developers) are slowed down in learning curve today



Snowpark Container Services: Extending the Benefits of Snowflake

WHAT IS IT

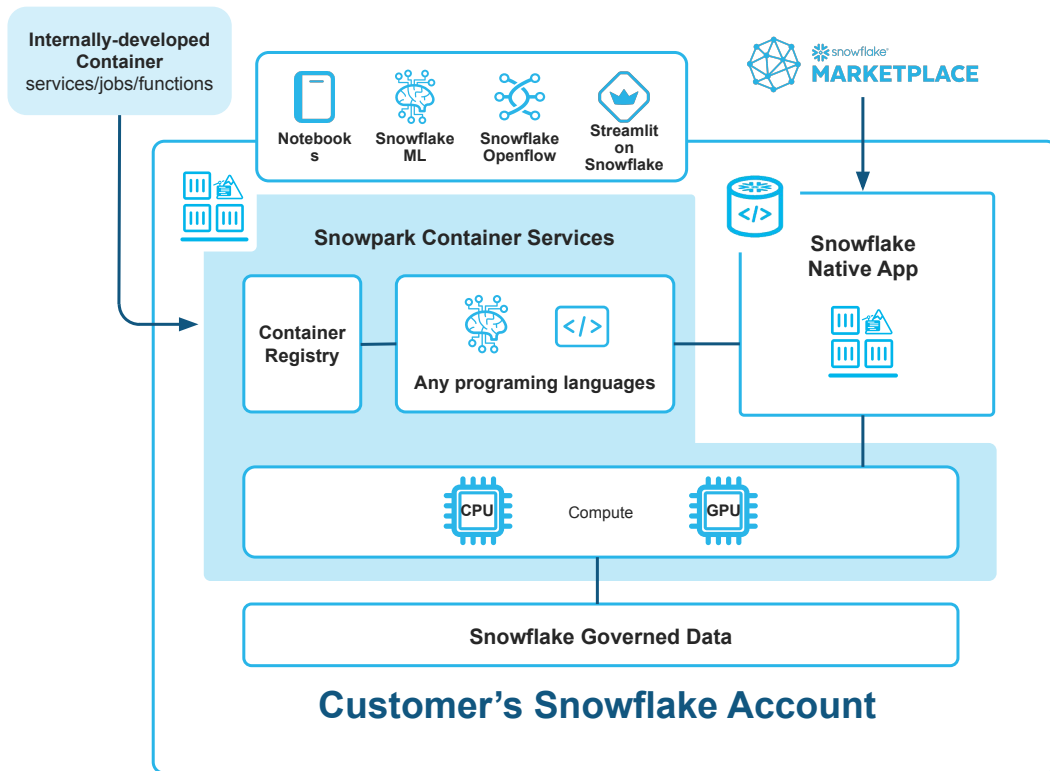
Snowpark Container Services (SPCS) is a powerful **extension** that turns Snowflake into a platform for running containers, services, and AI/ML apps securely and natively alongside your data.

WHY USE IT

- Language & hardware freedom
- One platform for Data + AI + Services
- Run advanced apps next to your data



Trusted by 1000+
customers, powering AI,
ML and data apps



Challenges and Benefits

Current Challenges

Heavy Refactoring of code

Running Workloads outside

Limited flexibility in languages, runtimes and libraries

Complexity of platforms like Kubernetes & Amazon SageMaker

Data Governance

Procurement for partner apps & connectors

Product Benefits

Bring Your Own Language (BYOL)

Expanded Compute - Large slew of instance types incl - GPU

Endpoints - Easily deployable App UI, API's, & UDF integration

No more Networking / Access headaches!

Lightweight Lifecycle APIs w/smaller manifest files

Data Governance - data never leaves your snowflake security boundary



The Core Value: Security, Governance, and Simplicity



Simplicity

Easy to use — for every developer/data scientist, not just infrastructure experts

VS

Complex K8s setup, networking, and IAM — slowing onboarding and dev.



Data & AI ready

Optimized for data-intensive workloads

VS

Slow data transfers, storage bottlenecks, and complex scaling for AI/ML and Apps



One Product

A single, integrated platform and deeply integrated across Snowflake (Streamlit, Cortex, SQL, Snowpark etc.)

VS

Stitching together dozens of products



The Core Value: Security, Governance, and Simplicity



Security/ Governance

Built-in controls for access, isolation, and compliance

VS

Manually configuring IAM, VPCs, policies, and audit controls



Elastic & Fast

Up to 10× faster startup vs. typical Kubernetes platforms

VS

Slow pod startup, cold starts, and rigid autoscaling



Bring Your Stack

Ultimate flexibility and extensibility inside Snowflake (Any language, framework, tools)

VS

Accessing and processing data needs lots of customization



SUPPORTED WORKLOADS



Modern Workloads Powered by Containers

Build. Extend. Run. Your Workloads



ML Workloads

Train and execute inference for large scale models using GPUs with any language including Python and R



Custom LLMs

Run open source LLMs, and third-party LLMs tuned to your data



Data Engineering

Run C/C++ workloads and web APIs that process large amounts of data



Applications

Run any component of an app from APIs to UIs

& more



Customer Solution - Bayer



> Description of Preview Customer Use Case

Customer is the largest Pharma company in the world who wants to surface sales insights and customer churn analysis to Sales leadership by deploying LLama2 inside of Snowpark Container Services and training on Sales data stores in Snowflake

> Stats From Preview Customer Usage

Took 15 days to deploy all the pieces to run LLama2 on EKS and still couldn't get legal approval to push real data. Took 30 minutes to deploy LLama2 on SPCS. Wait time for getting GPU3 and GPU7 is 6-8 weeks from AWS

“The only place we can run LLM for Bayer is on Snowpark Container Services both because of ease of use, GPU availability and Data Governance”

- **Jason Sypniewski**
Director, Data Engineering & ML, Bayer

What Are the Key Attributes of an Ideal Customer or Use Case?

- Customer has issues procuring GPUs from CSPs
- Data needs to stay within the Snowflake security perimeter
- Seeing issues with data going outside or even procuring GPUs



Customer Solution - Sailpoint

➤ Description of Preview Customer Use Case

Customer is an identity management solution running very complex ML algorithms using CPU inside/outside of Snowflake taking 15mins. Running in Snowpark container Services closer to the data with GPU the algorithm job was able to be run in 3min (5x improvement).

➤ Stats From Preview Customer Usage

5x time improvement from running inside Snowpark Container Services with GPU vs. outside GPU

“Not much work except running Python insides Snowpark Container Services. So far I am impressed! I worked with EKS for 7 years and this is much easier to get GPU running than in EKS”

- **Kelly Grizzle**
Sr. Software Architect, SailPoint

What Are the Key Attributes of an Ideal Customer or Use Case?

- Customer workload or solution that interact directly with Snowflake data
- Data needs to stay within the Snowflake security perimeter
- Seeing issues with data going outside or with compute running on Snowpark



Partner Solution - Hex



> Description of Preview Customer Use Case

Hex is a hosted IDE solution and partner of Snowflake. Before customer had to go to the Hex website to deploy IDE to work with Snowflake in EKS. With Snowpark Container Services customer will be able spin up Hex and connect securely to their data without ever leaving the Snowflake security perimeter

> Problem Solved with Preview Customer Usage

Customer data did not have to leave Snowflake to achieve the benefits of a hosted IDE experience

“I am impressed how quickly you guys shipped Snowpark Container Services and how easy it is to use and it just works! We got the whole application working inside in a very short time”

- **Caitlin Colgrove**
CTO and co-founder of Hex

What Are the Key Attributes of an Ideal Customer or Use Case?

- Partner solution was running outside
- Customer love partner solution but want Data to stay within the Snowflake security perimeter



ARCHITECTURE



The SPCS Architecture: A Unified Control Plane



Defined via 2 Artifacts



Managed & Abstracted Infrastructure



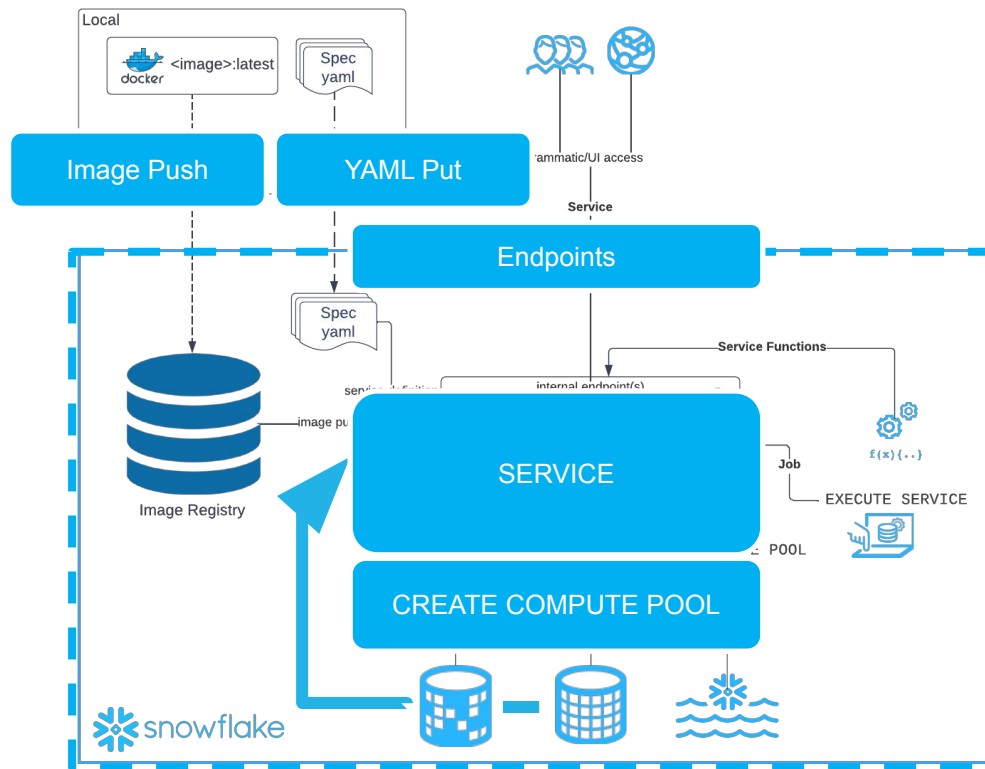
Direct & Secure Data Access



Configurable Interfaces



On Demand



User Model: Key Concepts

Image Registry - OCIv2-compliant image registry service hosted by Snowflake.

```
$ docker login myorg-myacc.registry.snowflakecomputing.com
```

Image Repository - Storage unit for container images.

```
create schema dev_db.test;  
create image repository dev_repo;
```

```
$ docker push  
myorg-myacc.registry.snowflakecomputing.com/dev_db/test/dev_repo/training
```

Compute Pool - Compute resources for container workloads. Analogous to warehouse for query workloads.

```
create compute pool dev_pool  
instance_family = standard_4  
min_nodes = 5  
max_modes = 5  
auto_resume = true;
```

User Model: Key Concepts

Container Services Specification - YAML configuration for a container application

```
spec:
  container:
    - name: server
      image:/dev_db/test/server
    - name: sidecar
      image:/dev_db/test/sidecar
  env:
    FOO: BAR
  volumeMounts:
    - name: vol1
      mountPath: /local/user/data
  endpoint:
    - name: callme
      port: 8080
  volume:
    - name: vol1
      source: @mystage
```


User Model: Key Concepts

Service/Jobs - a containerized application implemented by Kubernetes resources (service, jobs, replica sets, volumes, secrets, etc.).

Service: A long running service that does not end on its own.

```
create service web_server
in compute pool dev_pool
from @dev_stage spec=cool_app.yaml
min_instances = 5
max_instances = 10
```

Job: A type of Snowflake job (in addition to stored procs, tasks, etc.) that invokes a single container execution.

```
execute service
in compute pool dev_pool
from @dev_stage spec=cool_app.yaml
```

Container Application Interface

- public endpoints (internet ingress)
- UDF (SQL invocation)
- in-cluster DNS (service-to-service communication)

User Model: Key Concepts

Service Class - an encapsulated factory and interface for creating services or jobs, which contains container services configurations and permissions, and manages version control.

```
create service class web_server_factory
from @dev_stage spec=cool_app.yaml
version = v1_0;

alter service class web_server_factory
add version v1_1
from @dev_stage spec=cool_app.yaml;

create service web_server from web_server_factory;
```

User Model: Key Concepts

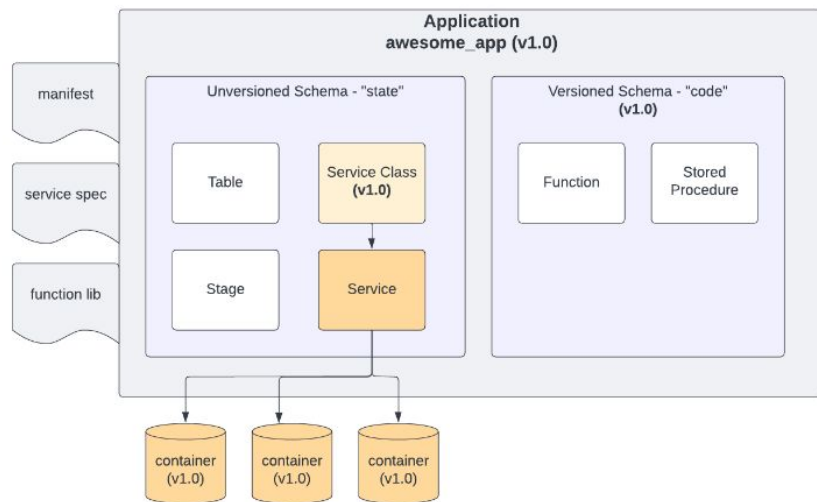
Native Apps - a sharing framework for Snowflake data and applications that can be backed by container services.

```
manifest_version: 1

version:
  name: V1
  label: "Version 1.0"
  comment: "The first version"

artifacts:
  setup_script: /scripts/setup.sql
  default_streamlit: ui.config_streamlit
  readme: reame.md
  container_services:
    images:
      - /provider_db/provider_schema/provider_repo/server:prod
      - /provider_db/provider_schema/provider_repo/web:1.0

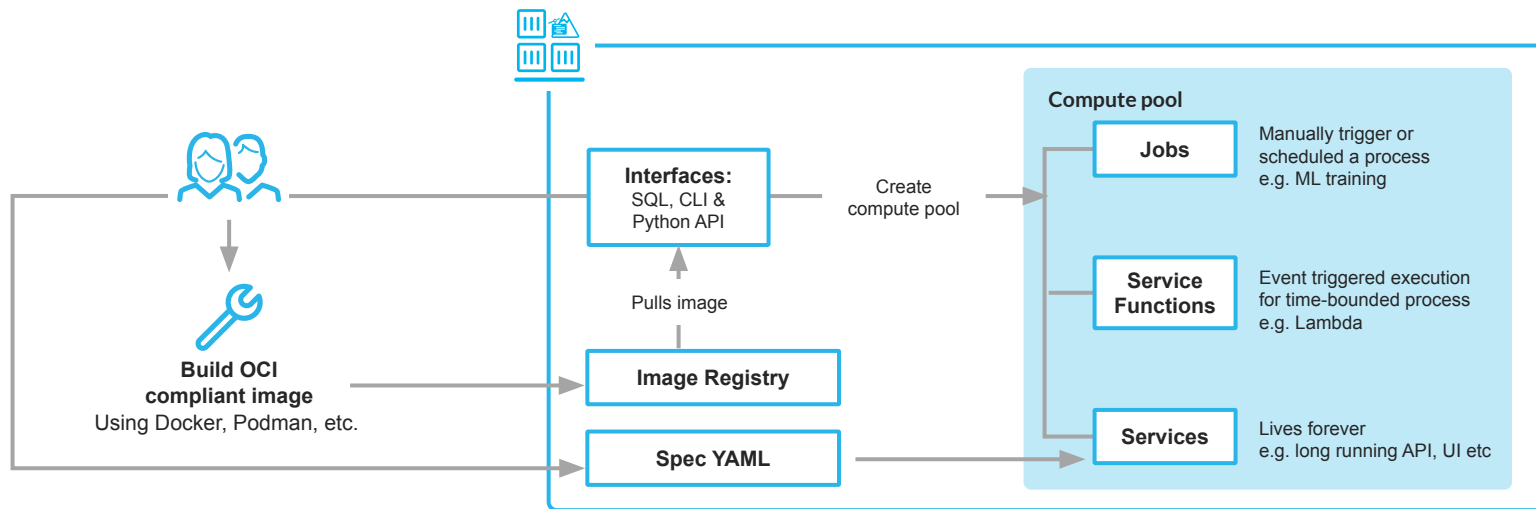
configuration:
  log_level: debug
  trace_level: on
```



DEPLOYMENT CYCLE



From Code to Cloud: The SPCS Deployment Lifecycle



Role-based access controls



Interfaces: SQL, CLI & Python API



Private Image Registry



12 Instance Types (CPU/HIGHMEM/GPU)



Secure Egress & Ingress



Service to Service Communication



Built-in observability



Core Building Blocks: Services, Jobs, and Functions

Image Repository

Snowflake's storage location for container images.

Schema-level constructs and resemble stages.

```
-- Schema-Level  
CREATE IMAGE REPO kafka_img_repo;
```

Compute Pools

Nodes where an application's services will run.

```
-- Account-Level  
CREATE COMPUTE POOL kafka_cp  
  INSTANCE_FAMILY=CPU_X64_XS;
```

Services

Actual running instances of the application within a compute pool.

Long Running Services

Stateful applications with constant uptime.

(e.g. Web UI, Kafka clusters, databases, etc.)

```
CREATE SERVICE kafka_svc  
  IN COMPUTE POOL kafka_cp  
  SPEC_FILE='config.yaml';
```

Job Services

Event-triggered time-bound jobs with minimal state.

(e.g. ML model training)

```
EXECUTE JOB SERVICE  
  IN COMPUTE POOL ml_compute_gpu  
  SPEC_FILE='config.yaml';
```

Service Function

Stateless user-defined function associated with a specific service.

(e.g. Cortex Search, Complete)

```
CREATE FUNCTION search_udf()  
  RETURNS float  
  SERVICE=search;
```



Creating Value: Defining Services, Jobs, and Service Functions

```
CREATE COMPUTE POOL my_cp1  
  MIN_NODES = 1  
  MAX_NODES = 3  
  INSTANCE_FAMILY = GPU
```

Define compute pool
size and instance type

```
CREATE SERVICE [ IF NOT EXISTS ] inference point  
  MIN_INSTANCES = 1  
  MAX_INSTANCES = 3  
  COMPUTE_POOL = my_cp1  
  SPEC = @my_service_stage/path/to/inference_point.yaml
```

Choose and define
execution as service,
function or job

Point to spec file with
service definition





SPCS vs AWS / Azure

Choosing the right container runtime for Snowflake-centric workloads



Why SPCS



- 1. Unified governance**
Data stays inside Snowflake's security perimeter



- 2. Simplified architecture**
No separate VPC, IAM, or service-account sprawl



- 3. Single billing**
One Snowflake bill and simpler cost visibility



- 4. Zero data egress fees**
Compute sits next to the data



- 5. Native RBAC integration**
Snowflake roles govern access

VS

Why AWS / Azure



- 1. More flexibility**
Better for multi-cloud orchestration and external services



- 2. Better discount economics at scale**
Reserved pricing and spot options can reduce long-running costs



- 3. Mature platform operations**
Strong fit for teams already standardized on managed cloud platforms



- 4. Broader infrastructure options**
Better for exotic hardware or specialized runtime needs

Cost trade-offs



Small workloads:
cost gap is usually modest



Bursty or moderate workloads:
SPCS often wins on lower operational overhead



Large always-on clusters:
AWS / Azure can be cheaper at scale

Best fit for SPCS



- ✓ Snowflake-centric teams
- ✓ Security-sensitive workloads
- ✓ Limited DevOps capacity
- ✓ POCs and mid-scale pipelines

Best fit for AWS / Azure



- ✓ Heavy multi-cloud orchestration
- ✓ Very large always-on clusters
- ✓ Existing managed platform investment
- ✓ Specialized hardware requirements



SPCS wins on simplicity, security, and operational efficiency.

AWS / Azure wins on flexibility, scale economics, and broader platform options.

DEMO



Resources

- Tutorials:
 - [Standard](#): create necessary objects, a service, a job service
 - [Advanced](#): network communication, storage configuration, advanced RBAC
- [Repository with sample applications](#)
- Quickstarts:
 - [Build a Data App and run it on Snowpark Container Services](#)
 - [Call Centre Analytics with Snowflake Cortex LLM and Snowpark Container Services](#)
 - [Show Compute Pool Instant Families](#)
- [Webinar](#)





Snowflake中国微信公众号

Q&A



Snowflake中国微信视频号